DAY 2 / MODULE 3

Continuous Integration with Azure Pipelines

Module 3 | Four-hour practical workshop

One application. Reviewed YAML. Repeatable validation.

Development, DevOps, QA, release engineering and SRE

What you will be able to demonstrate

01 Author YAML that restores, lints, tests and publishes results and coverage.

02 Predict CI, PR and scheduled execution and inspect a run’s timeline.

03 Publish an immutable 1.0.N output to Pipeline Artifacts and an Azure Artifacts feed.

04 Use a parameterised template, variable group and explicitly mapped secret.

One change, several engineering perspectives

Role	Question to answer with evidence
Developer	Did my commit build and pass the relevant tests?
QA	Which tests ran, failed or were skipped?
DevOps / platform	Are execution and configuration repeatable?
Release engineering	Which exact package belongs to this run?
SRE / technical lead	What can this result prove, and what is still unverified?

Readiness before the first run

01 Open your Azure DevOps project and the orders-api repository.

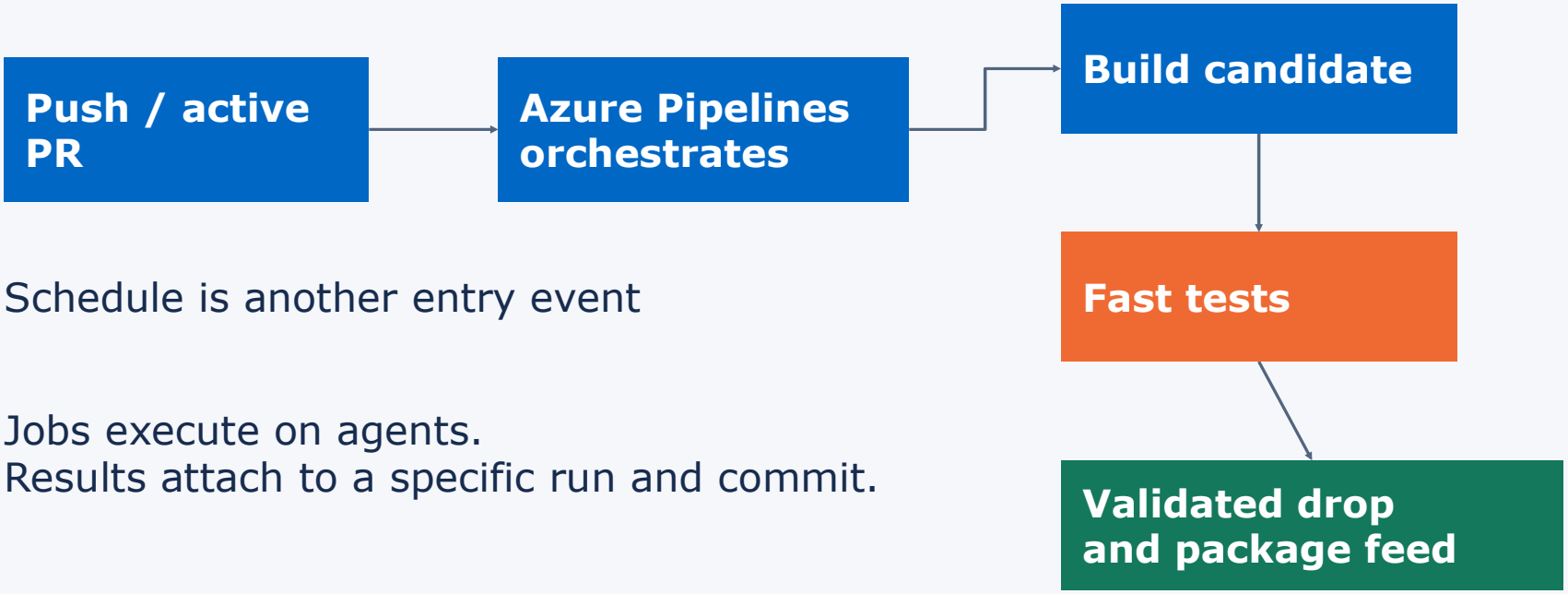
02 Confirm Basic access, pipeline creation rights and branch-policy editing rights.

03 Verify a Microsoft-hosted job actually runs, plus buddy reviewer access.

04 Have Python 3.12, Git and the lab files ready on the learner VM.

05 For the feed lab, prepare orders-feed and publishing permission for the build identity.

The CI evidence flow



Schedule is another entry event

Jobs execute on agents.
Results attach to a specific run and commit.

Why YAML for this programme

Concern	YAML pipeline	Classic editor
Review	PR reviews the pipeline change	Configuration edited through UI
History	Git diff, commits and branches	Separate pipeline revision history
Reuse	Step/job/stage and extends templates	Task groups in supported Classic workflows
Branch experiments	YAML can evolve on a branch	Configuration is separate from source branches

YAML describes the work, agents execute it

```
trigger:
  branches:
    include: [main]
pool:
  vmImage: ubuntu-24.04
steps:
- checkout: self
- script: |
    echo "Build: $BUILD_BUILDID"
    echo "Commit: $BUILD_SOURCEVERSION"
  displayName: Identify this run
```

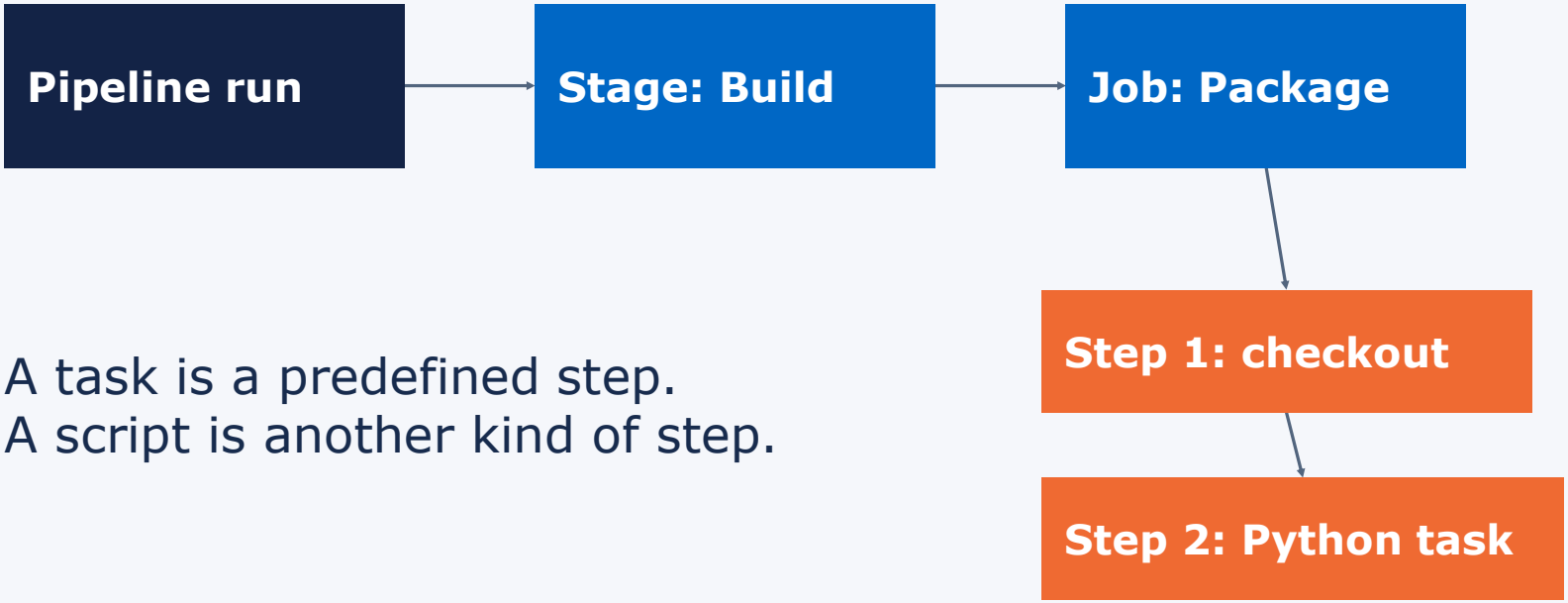
01 Start with one job and two steps.

02 The checkout retrieves the source revision.

03 A Bash script returns an exit code.

04 Commit YAML through the Day 1 PR process.

Pipeline, stage, job, step and task



A task is a predefined step.
A script is another kind of step.

Stages coordinate work. Jobs allocate execution. Steps run sequentially inside a job.

LAB 1 · CORE TASK

Lab 1: create and inspect the first pipeline

01 Create branch ci/module3 and edit azure-pipelines.yml.

02 Use the minimal YAML, save, commit and push.

03 Select Pipelines > New pipeline > Azure Repos Git > existing YAML if needed.

04 Run the pipeline against your branch and open the job logs.

05 Record the run ID, source branch, commit and selected image.

GUIDED PORTAL TASK · 3 MIN

Run summary, logs and timeline

01 Summary: confirm run number, branch, commit and trigger reason.

02 Stage/job view: find failed, skipped and queued work.

03 Step logs: find the first causal error and command exit code.

04 Tests and coverage: inspect evidence rather than only the green icon.

05 Artifacts: find the exact output and its source run.

Stages establish dependencies

```
stages:
- stage: Build
  jobs:
  - job: Package
    steps:
    - script: echo "Build placeholder"
- stage: Test
  dependsOn: Build
  jobs:
  - job: FastTests
    steps:
    - script: echo "Test placeholder"
```

01 Place pool at pipeline level.

02 Replace the previous root steps list.

03 Test waits for Build by default.

04 These echo commands are placeholders.

Events that start validation runs

Event	Configuration	What to inspect
Push	trigger in YAML	Branch and changed paths
Active Azure Repos PR	Build-validation policy on target	PR merge revision and policy result
Clock	schedules in root YAML	UTC cron, branch and always setting
Manual run	Run pipeline	Selected branch and parameters

Explicit branches and path filters

```
trigger:
  batch: true
  branches:
    include:
    - main
    - feature/*
    - ci/*
  paths:
    exclude:
    - docs/**
    - README.md
```

01 batch groups changes while CI runs.

02 An eligible branch AND path are needed.

03 Paths are relative to the repository root.

04 These filters do not configure PR policy.

PR validation checks the integration result



Policy belongs to the target branch

Azure Repos PR validation is configured in branch policy, not in a YAML pr: trigger.

Scheduled validation uses UTC

```
schedules:
- cron: '0 2 * * 1-5'
  displayName: Weekday validation
  branches:
    include: [main]
  always: true
```

01 02:00 UTC is 07:30 IST.

02 always: true runs without new changes.

03 Schedules belong in the root YAML.

04 Inspect Scheduled runs instead of waiting.

TRAINER DEMO

Repository-specific PR triggers

```
# GitHub or Bitbucket Cloud example only
pr:
  branches:
    include: [main]
  paths:
    exclude: [docs/**]
```

01 Azure Repos: target-branch build policy.

02 GitHub/Bitbucket Cloud: YAML pr: support.

03 Keep host-specific configuration explicit.

04 Do not paste this into the Azure Repos lab.

GUIDED TASK · 4 MIN

Manual runs with typed parameters

```
parameters:
- name: pythonVersion
  displayName: Python version
  type: string
  default: '3.12'
  values: ['3.11', '3.12']
trigger: none
pool:
  vmImage: ubuntu-24.04
steps:
- task: UsePythonVersion@0
  inputs:
    versionSpec: ${{ parameters.pythonVersion }}
- bash: python --version
```

01 Use examples/manual-parameters.yml.

02 Select Run pipeline and choose 3.11.

03 Repeat with 3.12 and compare logs.

04 Parameters are not secret variables.

TRAINER DEMO / EXTENSION

Pipeline-completion triggers

```
trigger: none
resources:
  pipelines:
  - pipeline: upstream
    source: orders-ci
    trigger:
      branches:
        include: [refs/heads/main]
pool:
  vmImage: ubuntu-24.04
steps:
- download: upstream
  artifact: drop
```

01 source is the pipeline definition name.

02 Authorise the pipeline resource.

03 Run after successful upstream main CI.

04 Use completion-consumer.yml from the kit.

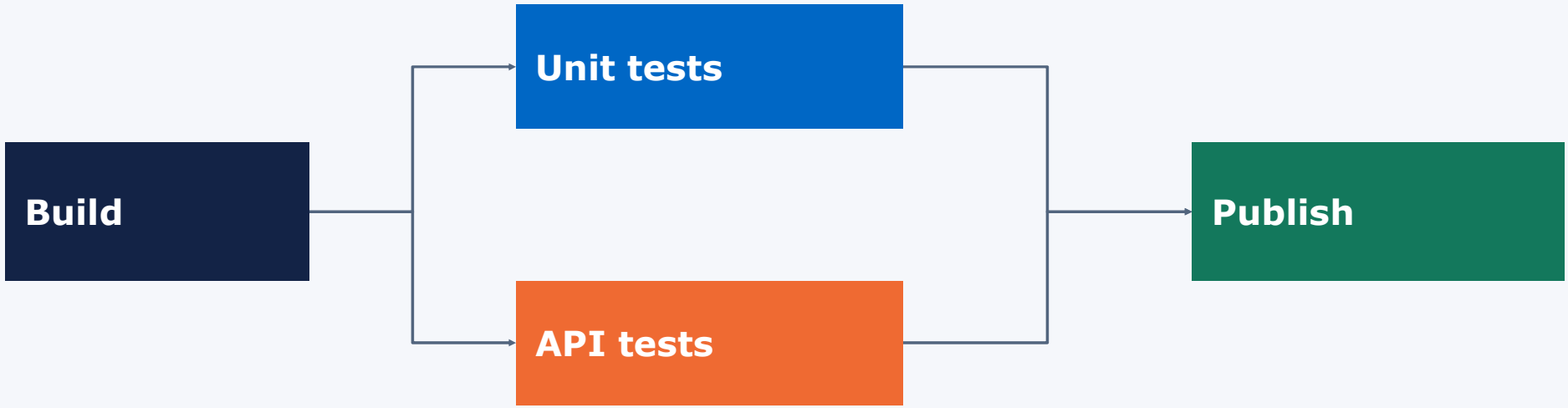
LAB 2 · CORE TASK

Lab 2: predict, push and observe

Experiment	Expected result
README-only change on ci/module3, no active PR	No branch CI from this trigger
orders.py change on ci/module3	Branch CI runs
Open active PR to main with required build policy	Separate PR validation runs
Push another code change to that PR	CI and PR validation may both run
Add weekday schedule to main through PR	Scheduled-runs view reflects schedule

20-MIN TOPIC BLOCK

Job dependencies and useful parallelism



Jobs can run concurrently only when the pool has enough parallel-job capacity.

Conditions define failure behaviour

Condition	Purpose in this lab	Cancellation caveat
succeeded()	Build/test-dependent publishing	Does not run after failure/cancel
succeededOrFailed()	Publish test evidence after failure	False after cancellation
always()	Attempt cleanup/reporting	Can evaluate true after cancel
and(succeeded(), ...)	Success AND a custom branch rule	Keep the success check explicit

GUIDED TASK · 4 MIN

Output from one job to another

```
jobs:
- job: Producer
  steps:
  - bash: |
      cmd='task.setvariable variable=ver'
      echo "##vso[$cmd;isOutput=true]1.0.7"
    name: stamp
- job: Consumer
  dependsOn: Producer
  variables:
    received: >-
      $[dependencies.Producer.outputs['stamp.ver']]
  steps:
  - bash: echo "$(received)"
```

01 Producer step needs a name.

02 isOutput=true exposes the value.

03 Consumer declares dependsOn.

04 Use a runtime expression to map it.

GUIDED TASK · 3 MIN

Timeouts and the cancellation window

```
jobs:
- job: FastChecks
  timeoutInMinutes: 10
  cancelTimeoutInMinutes: 2
  steps:
  - bash: python -m pytest -q
    timeoutInMinutes: 5
  - bash: echo 'Cleanup attempt'
    condition: always()
```

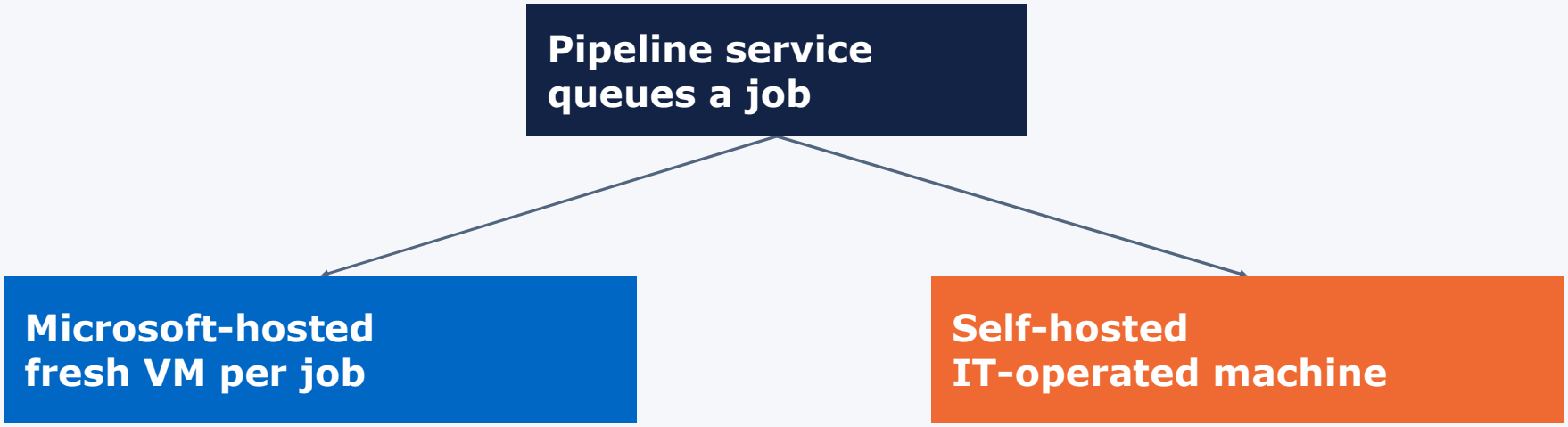
01 Job timeout bounds the whole job.

02 Step timeout bounds that command.

03 Service/agent limits still apply.

04 Cleanup is best effort.

Hosted and self-hosted execution



Managed image
Explicit restore each job
Hosted parallel-job capacity

Private-network reachability
Patch, isolate and clean
Manage your own capacity

Agent choices and operating responsibility

Choice	Control / operation	Use when
Microsoft-hosted	Fresh managed VM and published images	Public dependencies and simple lab setup
Self-hosted	Your team patches tools, OS and workspaces	Private network or specialised tooling
Managed DevOps Pools	Managed provisioning with configurable images/network	Managed fleet with tailored requirements

Image updates, capacity and minutes

01 Use ubuntu-24.04 explicitly for the lab.

02 The ubuntu-latest transition to 26.04 is a staged rollout beginning 19 October 2026.

03 An image label fixes the OS family, not every installed tool version.

04 Enabled free private-project hosted capacity: one job, 60 minutes per job, 1,800 minutes/month.

05 Check the organisation’s actual entitlement and current agent-image release notes.

LAB 3 · CORE TASK

Lab 3: locate the execution machine

```
- task: UsePythonVersion@0
  inputs:
    versionSpec: '3.12'
- bash: |
    uname -a
    python --version
    echo "Agent: $AGENT_NAME"
    echo "Sources: $BUILD_SOURCESDIRECTORY"
    echo "Workspace: $PIPELINE_WORKSPACE"
  displayName: Inspect execution context
```

01 Add under an agent job’s steps.

02 Read the agent details in the run.

03 Predict what a new job retains.

04 Do not install a self-hosted agent here.

10 MINUTES

Break and checkpoint

First checkpoint: your YAML runs on the intended agent.

You can distinguish push CI from required PR validation.

After the break: replace echo commands with build and test work.

Restore, build and test are different operations

Operation	Python lab example	Evidence
Restore dependencies	pip install -r requirements-dev.txt	Installed versions and success/failure
Build / package	compileall and python -m build	Syntax check and wheel
Test	pytest tests/unit tests/api	Assertions and JUnit results
Archive	Copy allowlisted files and zip	Application ZIP with version metadata

Repeatable dependency installation

```
- task: UsePythonVersion@0
  inputs:
    versionSpec: '$(pythonVersion)'
- bash: |
    set -euo pipefail
    python -m pip install --require-hashes \
      -r requirements.lock.txt
    python -m pip check
  displayName: Restore dependencies
```

01 Use python -m pip for the selected interpreter.

02 Hash-lock includes runtime and training tools.

03 Pin tested versions in the lab kit.

04 Verify the lock on the target image.

Restoring private packages through a feed



Authentication supplies credentials. Feed permissions and upstream configuration control access.

TRAINER DEMO / EXTENSION

PipAuthenticate for private dependency restore

```
- task: PipAuthenticate@1
  inputs:
    artifactFeeds: '$(System.TeamProject)/$(feedName)'
    onlyAddExtraIndex: false
- bash: |
    set -euo pipefail
    python -m pip install -r requirements.txt
  displayName: Restore using approved feed
```

01 Configure a PyPI upstream on the feed.

02 Grant the build identity access.

03 Authenticate before invoking pip.

04 Do not echo authenticated index URLs.

Caching reuses downloads, not correctness



A successful job can save a new cache. A cache hit alone does not prove correct dependencies.

A pip cache keyed by dependencies

```
- task: Cache@2
  inputs:
    key: >-
      "pip" | "$(Agent.OS)" |
      "$(pythonVersion)" | requirements.lock.txt
    restoreKeys: |
      "pip" | "$(Agent.OS)" | "$(pythonVersion)"
    path: $(PIP_CACHE_DIR)
    cacheHitVar: PIP_CACHE_HIT
- bash: >-
    python -m pip install --require-hashes
    -r requirements.lock.txt
  displayName: Install after cache restore
```

01 Define PIP_CACHE_DIR under Pipeline.Workspace.

02 Quoted key segments are literals.

03 Lockfile content determines the key.

04 A fallback hit is not an exact match.

Lockfiles and the restore boundary

01 A complete lock records transitive dependencies as well as direct ones.

02 Hash requirements.lock.txt in the cache key when the project uses it.

03 For a hash-locked pip file use pip install --require-hashes -r requirements.lock.txt.

04 Restore once per job environment and share the pip download cache between runs.

05 A second fresh hosted job needs its own installation or an explicitly prepared environment.

LAB 4 · CORE TASK

Lab 4: restore, cache and create a candidate

01 Add Python selection, dependency restore and Cache@2 to Build.

02 Run syntax validation on app.py and orders.py.

03 Write VERSION from the 1.0.N run number and build a wheel.

04 Copy approved application files and create orders-1.0.N.zip.

05 Compare cache logs and record an exact or fallback hit.

GUIDED TASK · 5 MIN

Linting checks code without executing tests

```
- bash: |
    set -euo pipefail
    python -m ruff check app.py orders.py tests
  displayName: Lint Python
- bash: python -m compileall -q app.py orders.py
  displayName: Syntax check
```

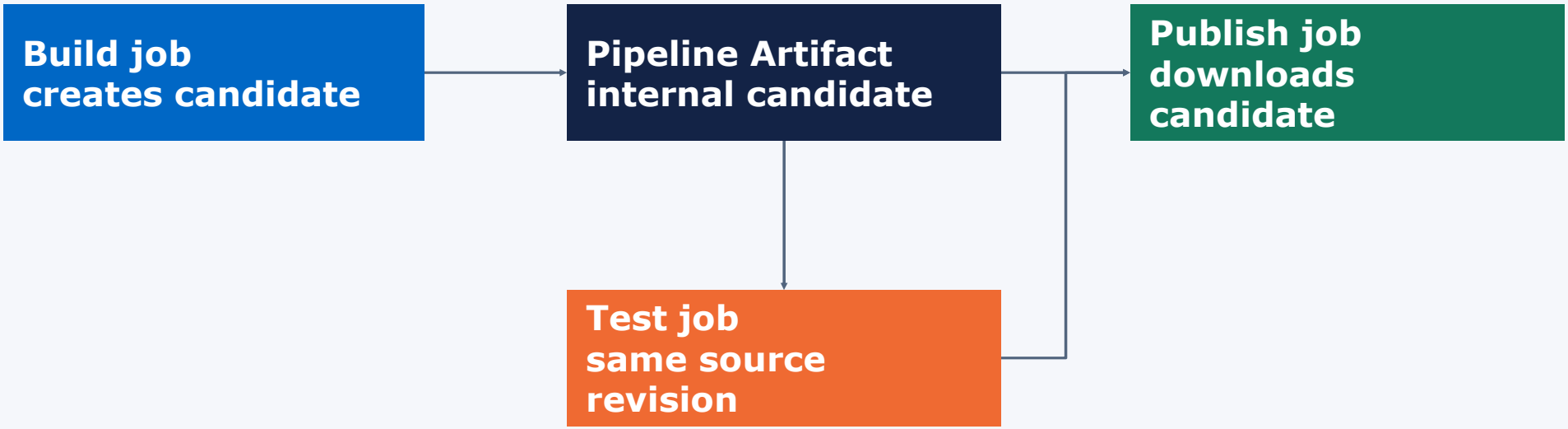
01 Ruff runs before packaging.

02 Use the repository’s agreed rule set.

03 Add an unused import and inspect failure.

04 Remove it and rerun.

Build once and move outputs explicitly



Separate agents have separate workspaces.

Publish waits for tests. The final drop reuses the candidate instead of rebuilding it.

Fast tests produce executable evidence

```
- bash: |
    set -euo pipefail
    mkdir -p test-results
    python -m pytest tests/unit tests/api -q \
      --junitxml=test-results/results.xml
  displayName: Run fast tests
```

01 Use the actual test paths in your repository.

02 pytest exits nonzero on failed assertions.

03 Do not append || true.

04 Do not enable continueOnError for required tests.

Coverage measures execution, not correctness

```
- bash: |
    set -euo pipefail
    mkdir -p test-results
    python -m pytest tests/unit tests/api -q \
      --junitxml=test-results/results.xml \
      --cov=orders --cov=app --cov-branch \
      --cov-report=xml:coverage.xml \
      --cov-report=html:htmlcov
  displayName: Tests and coverage
```

01 JUnit describes test outcomes.

02 Coverage XML describes exercised code.

03 Branch coverage reveals untested paths.

04 Threshold enforcement is Module 4.

Publish results even after a failed test

```
- task: PublishTestResults@2
  condition: succeededOrFailed()
  inputs:
    testResultsFormat: JUnit
    testResultsFiles: test-results/results.xml
    failTaskOnFailedTests: true
    failTaskOnMissingResultsFile: true
    testRunTitle: Fast tests $(Build.BuildId)
```

01 The previous pytest step still fails the job.

02 The publisher makes failures inspectable.

03 Missing output is treated as a failure.

04 Do not apply this condition to package promotion.

Publishing coverage into the run

```
- task: UseDotNet@2
  condition: succeededOrFailed()
  inputs:
    packageType: sdk
    version: '8.0.x'
- task: PublishCodeCoverageResults@2
  condition: succeededOrFailed()
  inputs:
    summaryFileLocation: coverage.xml
    pathToSources: $(Build.SourcesDirectory)
    failIfCoverageEmpty: true
```

01 Install the task’s documented prerequisite.

02 Run after pytest creates coverage.xml.

03 Inspect the Code Coverage tab.

04 A missing report is missing evidence.

LAB 5 · CORE TASK

Lab 5: prove a test blocks the PR

01 Create a linked bug branch and add a boundary test: 1000 must yield 900.

02 Introduce or retain the faulty > comparison, then push and open an active PR.

03 Inspect the failed policy run and the published assertion failure.

04 Change > to >=, run tests and push the fix.

05 Ask the buddy to review the latest revision and confirm validation passes.

A failure is evidence to investigate

Symptom	Likely area	First evidence
YAML rejected before queue	Structure or schema	Line, indentation and editor validation
Queued without starting	Capacity / pool / permissions	Parallel jobs and resource authorisation
ModuleNotFoundError	Dependency or interpreter	Restore log and python version
Expected 900, actual 1000	Business logic or expectation	Assertion plus acceptance criteria
No test results	Path / runner / setup	pytest log and output file location
Feed upload returns 403	Pipeline identity permissions	Feed role and build-service identity

Two different artifact services

	Pipeline Artifacts	Azure Artifacts feed
Purpose	Files attached to a pipeline run	Versioned packages for consumers
Lab example	orders-1.0.N.zip and a wheel	Universal Package orders-api-drop 1.0.N
Identity	Run ID, commit, artifact name	Package name and version
Publishing	publish shortcut / task	UniversalPackages task; optional wheel upload
Lifecycle	Tied to run retention	Feed/package retention policies

Semantic run numbers with counter()

```
name: 1.0.$(patch)
variables:
- name: patch
  value: $[counter('orders-ci-1.0', 1)]
steps:
- bash: |
    printf '%s\n' "$BUILD_BUILDNUMBER" > VERSION
  displayName: Stamp package version
```

01 First run for this prefix is 1.0.1.

02 Counter scope is the pipeline definition.

03 Failed and PR runs also consume numbers.

04 Gaps are normal, never reuse versions.

Version and publish the validated output

```
- stage: Publish
  dependsOn: Test
  jobs:
  - job: Outputs
    steps:
    - download: current
      artifact: candidate
    - publish: $(Pipeline.Workspace)/candidate
      artifact: drop
      displayName: Publish validated build output
```

01 candidate was produced in Build.

02 Test depends on successful Build.

03 Default success condition gates Publish.

04 Download drop from the successful run.

CORE PUBLISH DESTINATION

Universal Packages publish arbitrary build files

```
- task: UniversalPackages@0
  inputs:
    command: publish
    publishDirectory: $(Pipeline.Workspace)/candidate
    feedsToUsePublish: internal
    vstsFeedPublish: >-
      $(System.TeamProject)/$(feedName)
    vstsFeedPackagePublish: orders-api-drop
    versionOption: custom
    versionPublish: $(Build.BuildNumber)
```

01 Same validated ZIP, no rebuild.

02 Universal package: arbitrary file bundle.

03 Python wheel: language package.

04 Use the main-only publishing condition.

Azure Artifacts feed setup

01 Open Artifacts and create the project-scoped orders-feed.

02 Grant the executing project Build Service Feed Publisher (Contributor).

03 Create lab-ci with pythonVersion, feedName and secret labToken.

04 Authorise only the intended pipeline to use the group.

05 Use the pipeline identity for upload, not a learner PAT in YAML.

OPTIONAL PYTHON PACKAGE EXTENSION

Authenticated Python package publishing

```
- task: TwineAuthenticate@1
  inputs:
    artifactFeed: $(System.TeamProject)/$(feedName)
- bash: |
    set -euo pipefail
    python -m pip install twine==6.1.0
    python -m twine upload \
      -r "$(feedName)" \
      --config-file "$(PYPIRC_PATH)" \
      "$(Pipeline.Workspace)"/candidate/dist/*.whl
  displayName: Upload wheel to feed
```

01 Run after downloading candidate.

02 Use Python selection in this job too.

03 Put these steps in a main-only job.

04 Publish the version in the downloaded wheel.

LAB 6 · CORE TASK

Lab 6: retrieve and verify both artifact destinations

01 Inspect candidate from Build and drop after passing tests.

02 Download the ZIP and read its VERSION file.

03 Confirm the run’s source commit matches the reviewed change.

04 On successful main CI, publish orders-api-drop as a Universal Package.

05 Download exact version 1.0.N from orders-feed and compare content.

Retention and exact-version retrieval

Object	Retention decision	Lab check
Pipeline run / artifacts	Project run retention and retained runs	Inspect policy and retain a reference run if authorised
Feed package version	Feed retention, protected/promoted versions	Review feed settings without deleting shared packages
Downloaded package	Consumer selects the exact version	Download orders-api-drop at 1.0.N
Deployment provenance	Keep source commit and package/run linkage	Confirm VERSION matches the producing run

Variables carry values, parameters shape YAML

Syntax	When used	Example
$(pythonVersion)	Macro before a task executes	Task input selecting Python
${{ parameters.pythonVersion }}	Template parsing / expansion	Typed template parameter
$[expression]	Runtime expression	Advanced calculated variable
Environment variable	Inside the process	BUILD_BUILDID in Bash

GUIDED TASK · 3 MIN

Variable scope and precedence

```
variables:
  label: pipeline
stages:
- stage: Check
  variables:
    label: stage
  jobs:
  - job: Scope
    variables:
      label: job
    steps:
    - bash: echo "$(label)"
```

01 Predict the output before running.

02 Nearest scope wins here: job.

03 Remove job override, then stage override.

04 Names should express purpose.

Variable groups centralise shared configuration

```
variables:
- group: lab-ci
- name: PIP_CACHE_DIR
  value: $(Pipeline.Workspace)/.pip

# lab-ci values in Pipelines > Library:
# pythonVersion = 3.12
# feedName = orders-feed
```

01 Authorise this pipeline in group permissions.

02 Do not give every pipeline access by default.

03 Map secrets into task environment explicitly.

04 Never echo a real secret as a lab exercise.

GUIDED TASK · 4 MIN

A secret variable mapped to a process

```
- bash: |
    python - <<'PYCODE'
    import os
    if not os.environ.get('LAB_TOKEN'):
        raise SystemExit('Secret is missing')
    print('Secret available to process')
    PYCODE
  env:
    LAB_TOKEN: $(labToken)
  displayName: Check secret presence
```

01 In lab-ci, add labToken and mark secret.

02 Use a harmless training-only value.

03 Authorise the intended pipeline.

04 Output presence, never the value.

CONCEPT ONLY · IMPLEMENT IN MODULE 6

Key Vault-linked groups: preview for Module 6

01 A linked variable group maps selected Key Vault secret names.

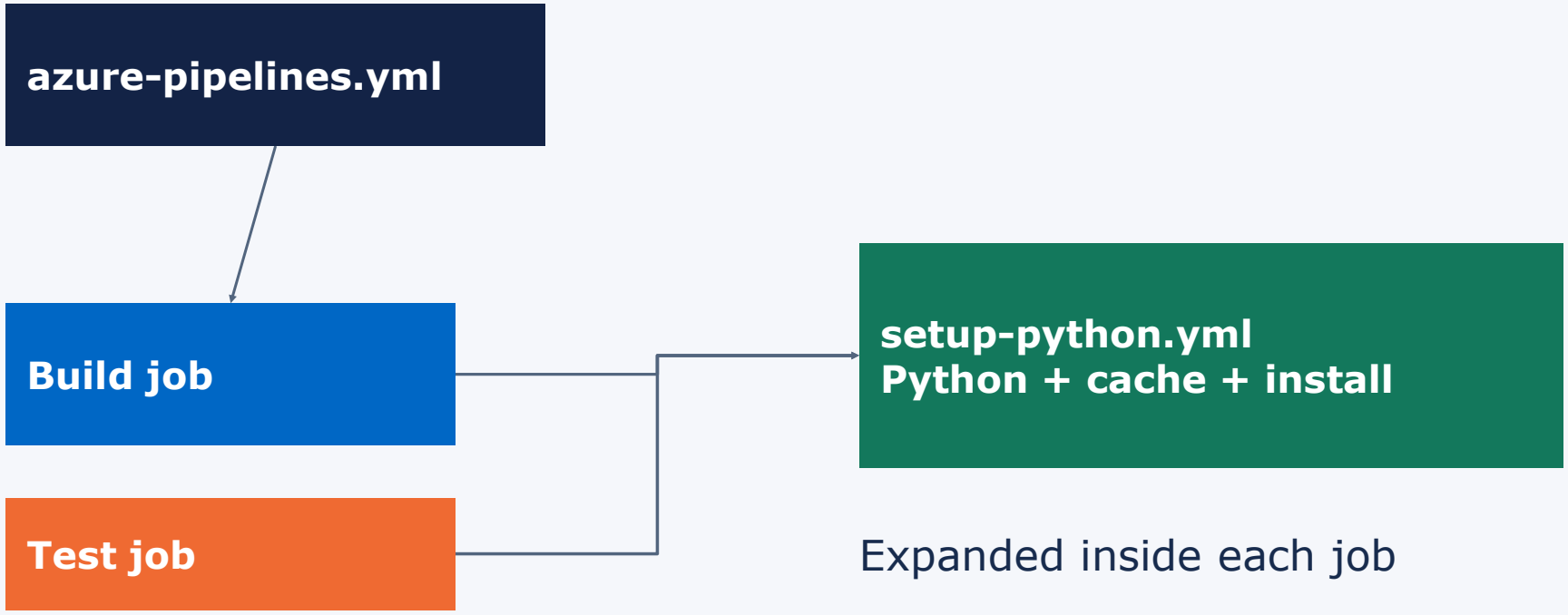
02 A service connection needs permission and network access to read them.

03 Pipelines still need authorisation to use the group.

04 Secret-value changes are retrieved when the pipeline runs.

05 Adding or removing secret names requires updating the group’s mappings.

Reusable setup belongs in a step template



Expanded inside each job

Template definition and invocation

```
parameters:
- name: requirementsFile
  type: string
  default: requirements.lock.txt
steps:
- task: UsePythonVersion@0
  inputs:
    versionSpec: '$(pythonVersion)'
- bash: >-
    python -m pip install --require-hashes
    -r "${{ parameters.requirementsFile }}"
```

01 Save as pipelines/templates/setup-python.yml.

02 Then call it under job steps:

03 - template: pipelines/templates/setup-python.yml

04 parameters:
requirementsFile: requirements.lock.txt

Step, job and stage templates

Template type	Called under	What it reuses
Step template	steps:	Python selection, cache and installation
Job template	jobs:	A complete job with pool/timeouts/steps
Stage template	stages:	A stage and its contained jobs
extends template	Root extends:	A governing pipeline structure

TRAINER DEMO / EXTENSION

A typed job template

```
# pipelines/templates/job-echo.yml
parameters:
- name: jobName
  type: string
  default: Inspect
jobs:
- job: ${{ parameters.jobName }}
  timeoutInMinutes: 5
  steps:
  - bash: echo 'Job template expanded'
```

01 Caller uses jobs: then - template:.

02 Stage template uses stages: instead.

03 Keep identifiers valid.

04 Run template-types.yml for both forms.

TRAINER DEMO / EXTENSION

Extends templates and governance

```
# examples/governed.yml
trigger: none
extends:
  template: ../pipelines/templates/governed.yml
  parameters:
    pythonVersion: '3.12'
```

01 Central template owns stage/job structure.

02 Expose approved, typed choices.

03 Protect the template repository and reviews.

04 Required-template checks enforce adoption.

LAB 7 · CORE TASK

Lab 7: remove duplication without hiding intent

01 Create lab-ci, authorise the pipeline and mark labToken secret.

02 Move Python selection, cache and installation into setup-python.yml.

03 Call the template from Build and Test with requirementsFile.

04 Map the secret into env and check presence without printing it.

05 Break a template path briefly, read the validation error and restore it.

10 MINUTES

Break before the integrated challenge

Save and push a working checkpoint.

Confirm the template and variable group are available.

After the break, demonstrate the whole workflow without bypassing policies.

LAB 8 · 15 MIN

Lab 8: one change, complete CI evidence

01 Create a linked change on a short-lived branch.

02 Show a deliberate test failure blocking the PR.

03 Fix the change and obtain review on the latest revision.

04 Complete the PR and inspect successful main CI.

05 Retrieve drop and verify Universal Package version 1.0.N.

Evidence checklist for the case study

Requirement	Evidence to show
Pipeline authored in Git	YAML/template commits and reviewed PR
Automatic CI and PR checks	Run reasons and required policy result
Dependencies restored and cached	Restore plus cache logs
Lint and coverage	Ruff output and Code Coverage report
Fast tests block poor changes	Failed run, corrected run, Tests tab
Identifiable outputs	Run ID, commit, 1.0.N, VERSION and drop
Reusable configuration	Template, lab-ci and secret-presence step
Azure Artifacts publishing	Feed package version on successful main run

Scenario check: green despite a failing assertion

01 A required test command is followed by “|| true”.

02 The pipeline is green even though one assertion failed.

03 Which change restores a meaningful gate?

04
A Increase retries
B Remove the suppressed exit code
C Rename the test job
D Make the policy optional

Scenario check: cache hit on a new agent

01 Cache@2 restores the pip download directory.

02 The next step skips pip install whenever a cache exists.

03 Why is that unsafe?

04
A The cache changes the branch policy
B A cache cannot store wheel files
C Cached downloads are not installed packages
D Python cannot run on hosted agents

What a successful CI run proves

01 The selected source revision passed the checks you actually configured.

02 The run produced identifiable build outputs plus test and coverage evidence.

03 It does not establish production health, complete test coverage or release approval.

04 Next: coverage thresholds, deeper quality gates and deployment controls.

APPENDIX

Reference: files and checkpoint recovery

File	Purpose
checkpoints/01-minimal.yml	First executable pipeline
checkpoints/02-build.yml	Single-job build/cache/test checkpoint
azure-pipelines.yml	Complete staged CI with package publishing
setup-python.yml (under templates)	Reusable setup, cache and dependency restore
README.md	Import, permissions, lab steps and troubleshooting
app.py, orders.py, tests, pyproject.toml	Standalone reference application and packaging
examples/*.yml	Triggers, job controls, private feeds and governance

APPENDIX

Reference: extension tasks by role

Role interest	Extension after core completion
Developer / QA	Add a boundary test and trace it to acceptance criteria
DevOps / platform	Add a template parameter and compare repeated-run cache logs
Release engineering	Inspect wheel metadata and package/run version relationship
SRE	Classify a restore failure versus an application failure
Experienced pipeline engineer	Design parallel jobs, then explain the effect of one-job capacity

TRAINER REFERENCE

Coverage map and execution depth

Area	Core hands-on	Demonstration / extension
YAML and triggers	Author, inspect, CI/PR/schedule	Completion trigger, runtime parameter run
Jobs and agents	Dependency, failed-job evidence	Outputs, timeouts, parallel queue, pool choices
Restore and testing	Lock/cache, lint, tests, coverage	Private feed and upstream restore
Artifacts	1.0.N drop and Universal Package	Python feed publishing, retention policy review
Variables/templates	Step template, group, secret env	Scopes, job/stage templates and extends

Reference: Microsoft documentation

01 Pipeline structure, Azure Repos triggers and agent execution

02 Scheduled triggers and pipeline caching

03 Test results and Pipeline Artifacts

04 Python package publishing and feed authentication

05 Variables, variable groups and templates